Referencia de Developer do ATC

Referência de developer do ATC

Esta referência complementa o Guia de Developer. O foco e pratico: como contribuir para o Advanced Trigonometry Calculator 2.1.9 sem introduzir regressoes.

Estrutura principal do projeto

- `Advanced Trigonometry Calculator/`: código C++ e ficheiros Visual Studio.
- `tests/`: regressão PowerShell, cobertura isolada e stress de memória.
- `docs/`: documentação Markdown, PDF e DOCX.
- `tools/docs/`: geradores de documentação.

Expectativas de código

- Manter nomes publicos de comandos estáveis salvo quando existir plano de compatibilidade.
- Preferir helpers existentes de parser, alocacao e formatacao.
- Manter comentarios de código em inglês.
- Evitar comportamento escondido que não possa ser testado ou documentado.
- Preservar o objetivo de compatibilidade Windows XP até Windows 11 onde o código atual o suporta.

Adicionar um novo comando

1. Localizar o handler mais próximo em `commands.cpp`, `main_processor.cpp` ou `main_aux_processor.cpp`. 2. Seguir o estilo atual de deteccao de comandos. 3. Manter prompts e outputs estáveis se forem testados. 4. Adicionar cobertura em `tests/run-atc-regression.ps1` quando possível. 5. Documentar o comando no guia e na matriz de cobertura.

Adicionar uma função matemática

1. Confirmar se pertence a `trigonometry.cpp`, `hyperbolic.cpp`, `logarithmic.cpp`, `statistics.cpp`, `digital_signal_processing.cpp` ou a outro módulo existente. 2. Considerar caminhos `double` e Boost `mp_float`. 3. Confirmar comportamento de modo angular se for trigonométrica. 4. Adicionar vocabulario de autocomplete se a função deve ser sugerida. 5. Adicionar testes de regressão com output estavel.

Adicionar um módulo guiado

1. Manter o fluxo interativo deterministico sempre que pratico. 2. Evitar abrir janelas externas durante testes automáticos. 3. Adicionar pelo menos um smoke test seguro. 4. Documentar lacunas manuais se a automação completa não for prática.

Adicionar um teste de regressão

Usar `tests/run-atc-regression.ps1` para verificacoes normais de comando/output. Os testes devem ser:

- deterministicos;
- independentes de caminhos absolutos locais;
- seguros com input redirecionado;
- isolados das settings do utilizador, ou com backup/restore pelo runner.

Não atualizar a contagem documentada de testes sem executar a suite.

Documentar uma funcionalidade

Quando uma funcionalidade muda:

1. atualizar primeiro a referência em inglês; 2. rever a versão portuguesa correspondente; 3. atualizar `tests/ATC_USER_GUIDE_COVERAGE.md`; 4. regenerar PDFs/DOCX quando relevante.

Riscos de regressão no parser e solver

Alteracoes no parser e solver devem ser testadas com:

- aritmética direta;
- multiplicacao implicita;
- variáveis;
- valores complexos;
- simplificacao de polinómios;
- `solve equation(...)`;
- `solver(...)`;
- modo de alta precisão quando relevante.

Evitar resolver um caso especial contornando o fluxo normal, salvo se esse comportamento for deliberadamente documentado e testado.

Compatibilidade Windows

O ATC contém comportamento de consola para ambientes Windows antigos e novos. Ter cuidado com:

- comportamento especifico do Windows Terminal;
- output ANSI/VT;
- tamanho e posicao da janela;
- comandos que abrem novas instancias, abas, ficheiros ou pastas.

Estabilidade de output

Os testes automáticos normalmente validam padroes de output. Antes de alterar texto visivel, considerar se elê aparece em:

- documentação de utilizador;
- notas de release;
- testes de regressão;
- relatorios exportados.

Checklist antes de commit/release

- Fazer build Release x64.
- Fazer build Release x86 quando houver alteracoes sensíveis a compatibilidade.
- Executar a suite de regressão.
- Executar cobertura isolada se a alteracao tocar comportamento interativo ou

source-level.

- Executar stress de memória para alteracoes com muitas alocacoes.
- Atualizar documentação e notas de release.

- Validar links Markdown e regenerar PDFs/DOCX quando necessario.